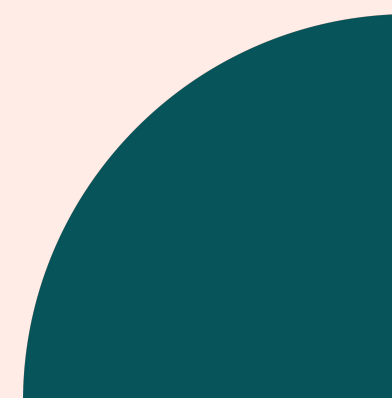


The Clean Code & Automation Cycle



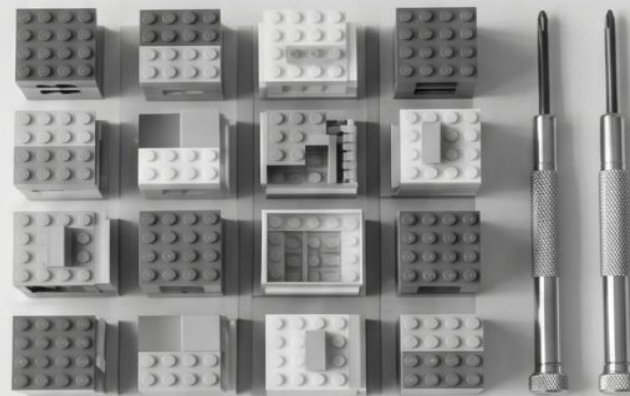
Development Environments

David Alonso Aguilar
Alberto Larios Espina



BEFORE

AFTER



01

INTRODUCTION

Project purpose

02

REFACTORIZATION

Fixing the code

03

CONCLUSION

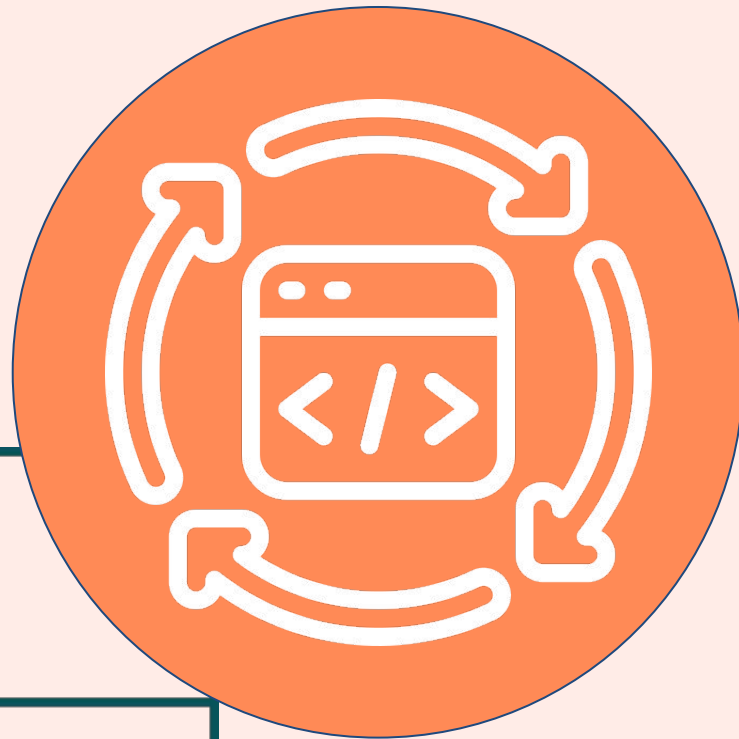
Conclusion of the refactorization



Introduction

01

Refactoring Project



We are automating a critical user journey on the SauceDemo platform. A user logs in, selects a backpack, and adds it to the cart. Our main goal isn't just making the test pass, but building a professional framework. To achieve this, we moved away from a single script to a structured Page Object Model, which allows us to manage web elements and test logic separately, making the entire suite more maintainable and easier to scale.



Introduction

Smelly Code



A monolithic script containing 10 specific code smells. It is fragile, difficult to maintain, and mixes setup, locators, and logic.

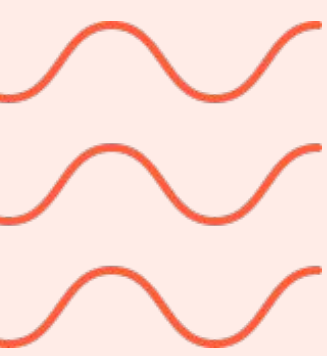
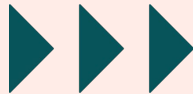
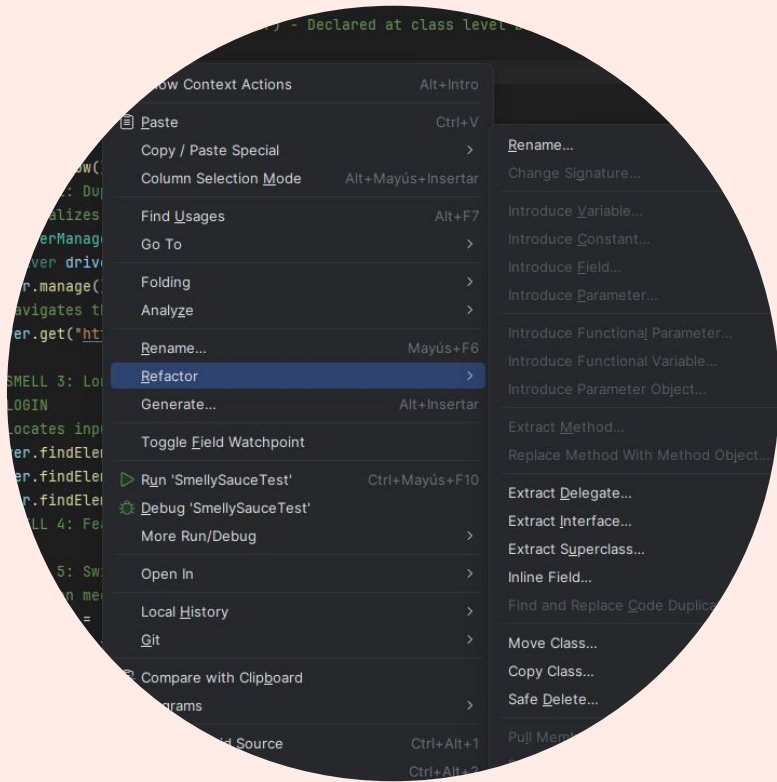
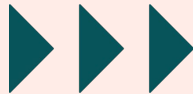
Clean Code



A scalable architecture using the Page Object Model (POM). Ensures high readability and easy maintenance.



Smelly Code Refactoring Clean Code



We got a functional but unoptimized code

We refactor to correct every smell

Clean and optimized code after refactorization



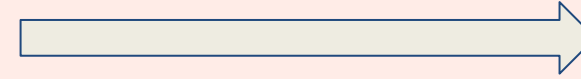
Refactorization

02



Refactorization

```
// SMELL 1: Temporary Field (OO Abuser) - Declared at class level but only needed locally  
no usages  
private String tempUser;
```

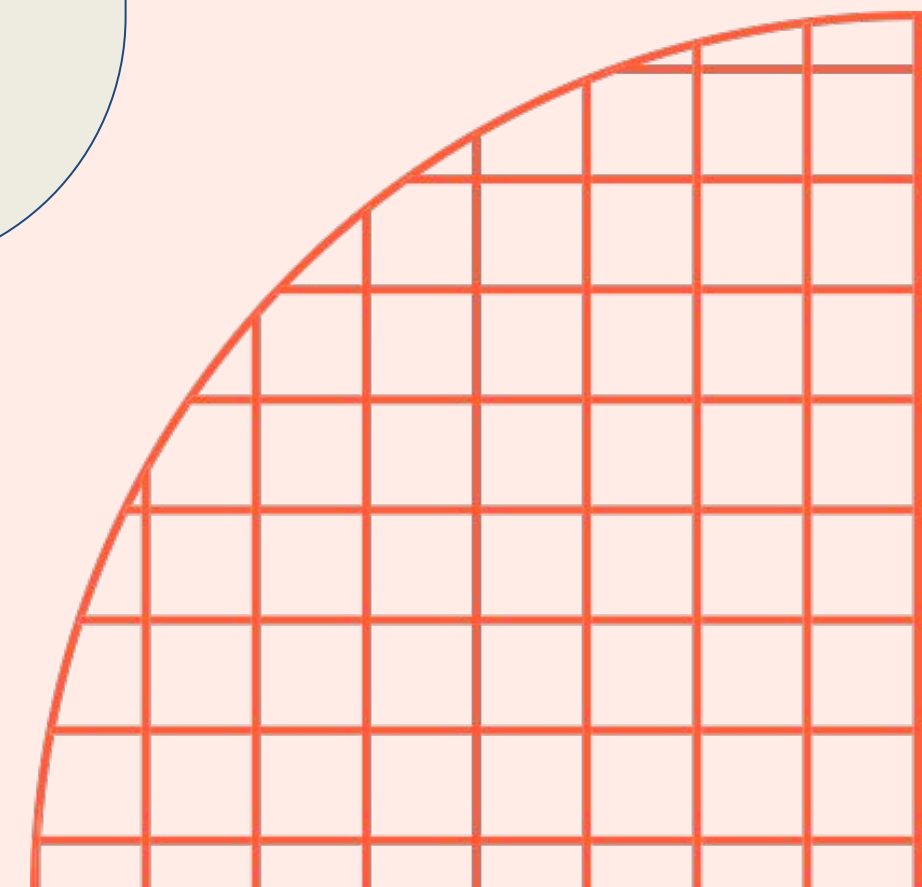
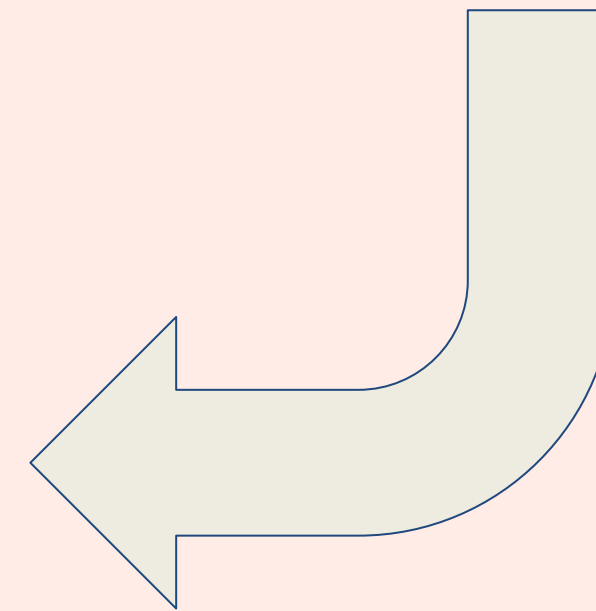


OO Abuser (Temporary Field):

The variable tempUser was declared at the class level but only used inside a specific method.

```
public void login(String user, String pass) {
```

In the refactored LoginPage, credentials are now passed as local parameters directly to the login(String user, String pass) method. This ensures that data only exists where it is actually needed, keeping the class state clean and professional.



Refactorization

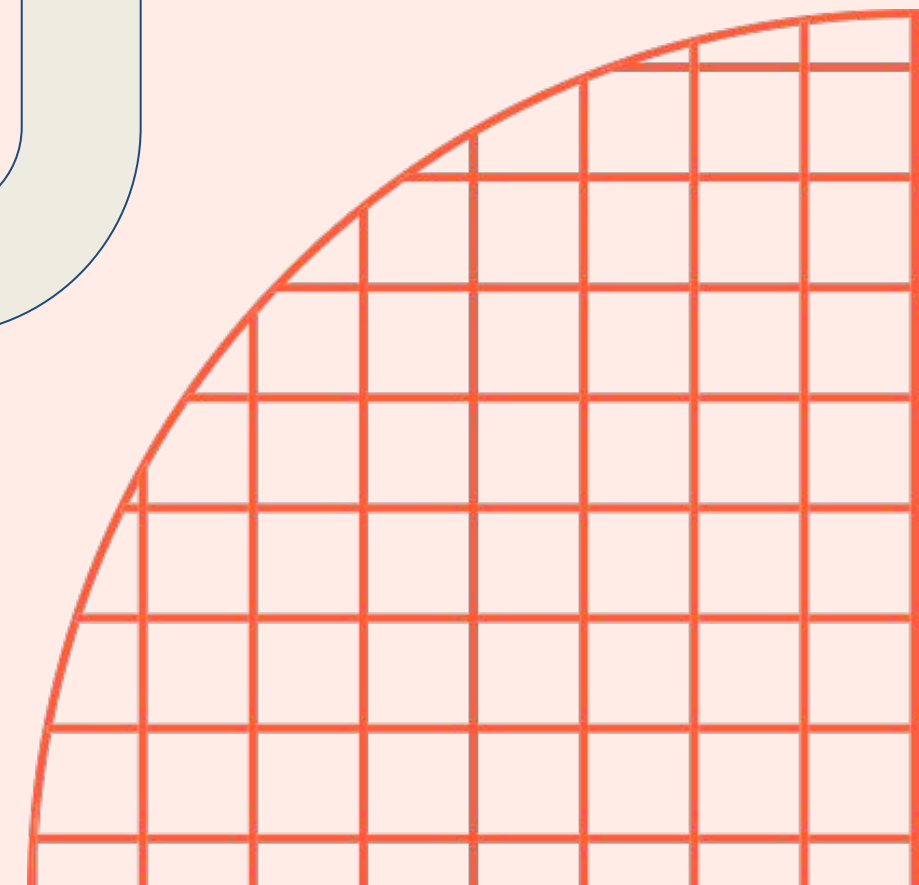
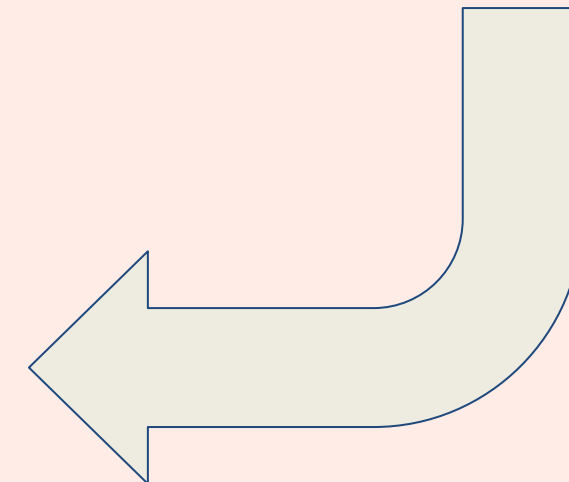
```
// SMELL 2: Duplicate Code (Dispensable) - Setup should be in @BeforeEach
// Initializes the ChromeDriver and opens the browser window
WebDriverManager.chromedriver().setup();
WebDriver driver = new ChromeDriver();
driver.manage().window().maximize();
// Navigates the browser to the target web application
driver.get("https://www.saucedemo.com/");
```



Dispensable (Duplicate Code):
Browser initialization and window management were coded directly inside the test method. This code would need to be repeated for every new test file.

We centralized the browser infrastructure into a BaseTest class. By using the @BeforeEach annotation, the setup runs automatically before every test without repeating code.

```
@BeforeEach  @ David Alonso Aguilar *
public void setUp() {
    WebDriverManager.chromedriver().setup();
    driver = new ChromeDriver();
    driver.manage().window().maximize();
}
```



Refactorization

```
void completeFlow() throws InterruptedException {  
    // SMELL 3: Long Method (Bloater) - This method is too long
```



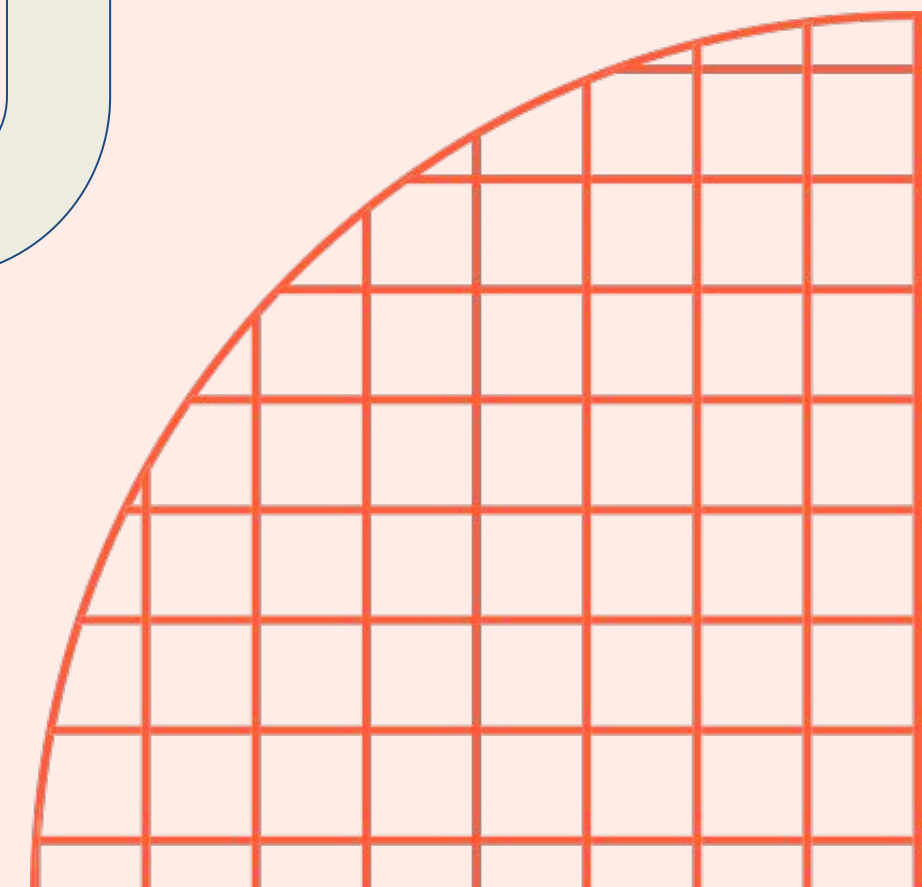
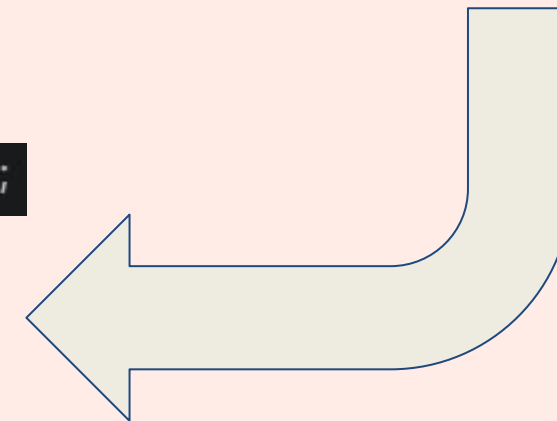
Bloater (Long Method):

The completeFlow() method was doing too many things: logging in, selecting products, and checking the URL. It was hard to read and maintain.

The logic was divided into small, descriptive methods within Page Objects, leaving the main test clean and readable.

```
loginPage.login( user: "standard_user", pass: "secret_sauce");
```

```
inventoryPage.addBackpackToCart();  
inventoryPage.goToCart();
```



Refactorization

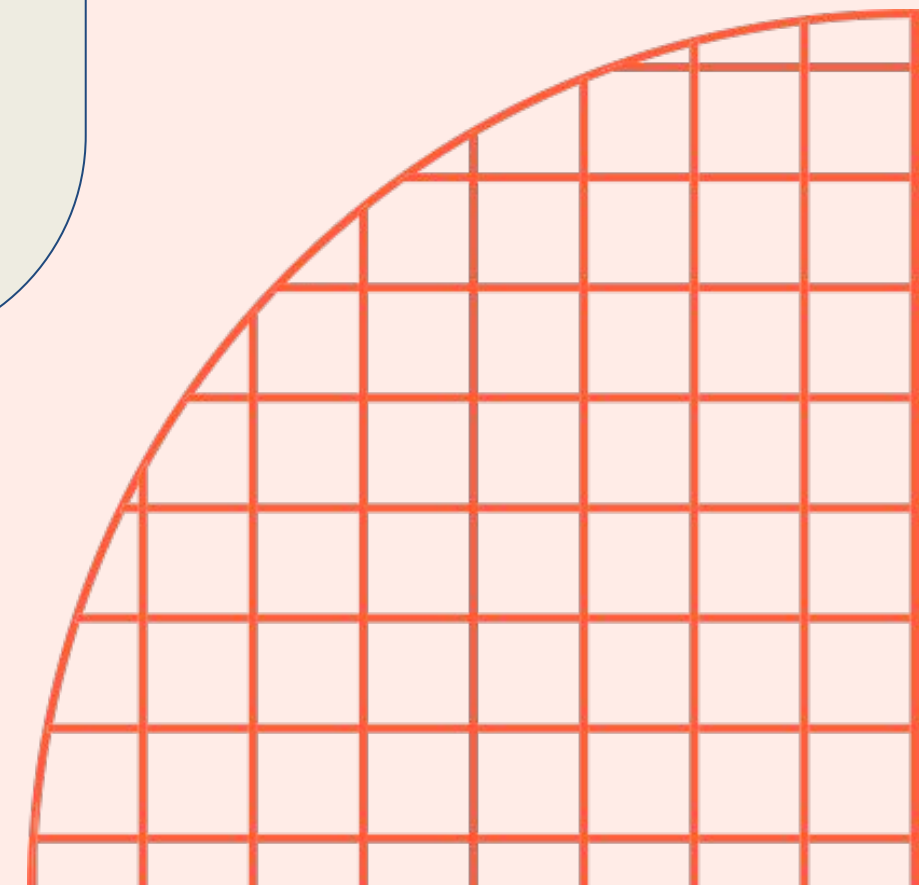
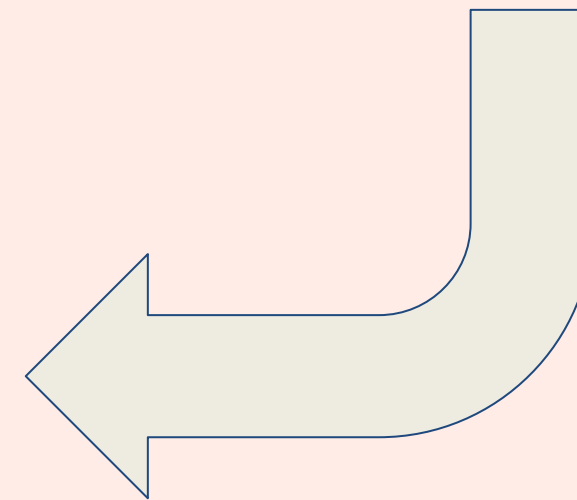
```
driver.findElement(By.id("user-name")).sendKeys(...keysToSend: "standard_user");  
driver.findElement(By.id("password")).sendKeys(...keysToSend: "secret_sauce");  
driver.findElement(By.id("login-button")).click();  
// SMELL 4: Feature Envy (Coupler) - Test is directly managing the page IDs
```



Coupler (Feature Envy):
The test class was directly interacting with IDs (like user-name) that belong to the web page logic. It was "envious" of the page's internal details.

```
public void login(String user, String pass) { 1usage  
    driver.findElement(userInput).sendKeys(user);  
    driver.findElement(passwordInput).sendKeys(pass);  
}
```

We moved the responsibility of interacting with web elements to the Page Objects. The test now only asks the page to perform a "business action" without knowing the technical IDs.



Refactorization

```
// SMELL 5: Switch Statement (OO Abuser) - Replace later to make the code more scalable
//Selection mechanism to decide which item to add to the cart
String item = "Backpack";
switch (item) {
    case "Backpack":
        driver.findElement(By.id("add-to-cart-sauce-labs-backpack")).click();
        break;
    case "Bike Light":
        driver.findElement(By.id("add-to-cart-sauce-labs-bike-light")).click();
        break;
}
```

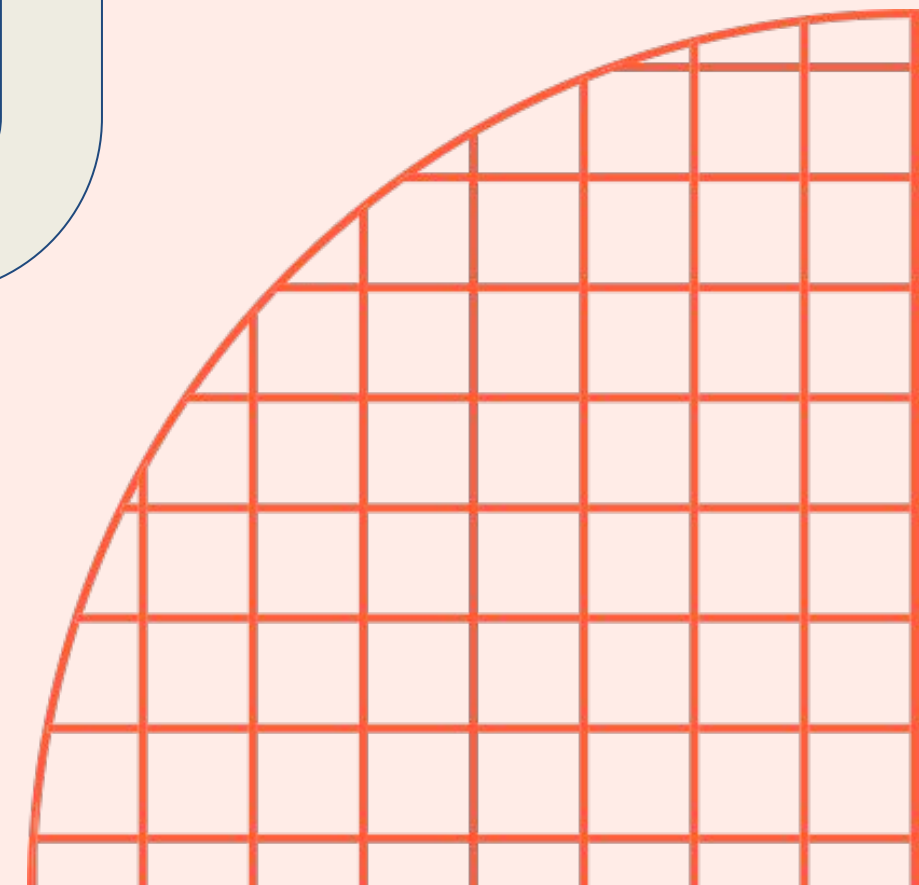
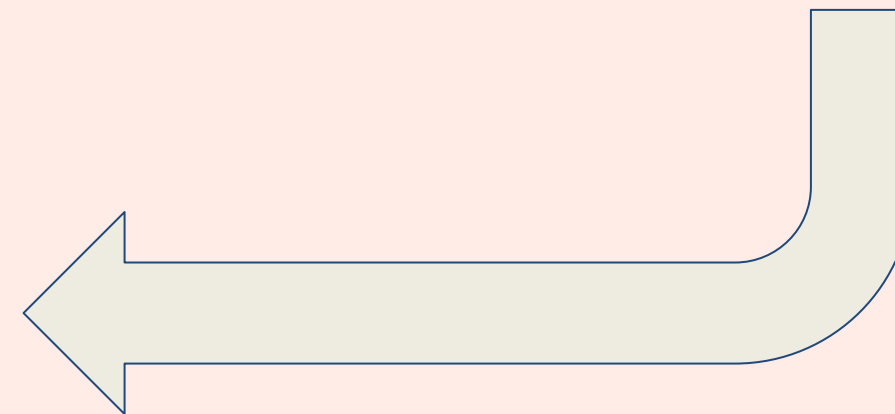


OO Abuser (Switch Statement):

A switch block was used to choose between products. This is rigid, adding a new product would require editing this complex logic every time.

```
public void addBackpackToCart() { driver.findElement(backpackAddButton).click(); }
```

We replaced the conditional logic with specific methods in the InventoryPage. This replaced conditional with polymorphism, making the code easier to scale.

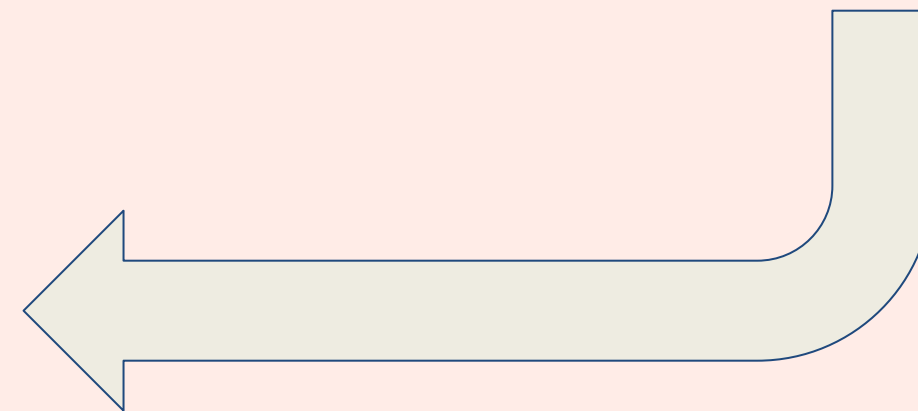


Refactorization

```
// SMELL 6: Dead Code (Dispensable) - Unused variable  
String unusedVariable = "I am not used";
```

Dispensable (Dead Code):
There were variables like
unusedVariable and extra code
that performed no function in the
final test.

We performed a clean purge of
the project. Any variable, import,
or method that did not contribute
to the test goal was deleted to
keep the workspace professional.



Refactorization

```
// SMELL 7: Divergent Change (Change Preventer) - Logic is mixed with ID,  
// Clicks the shopping cart icon to view selected items  
driver.findElement(By.className("shopping_cart_link")).click();
```

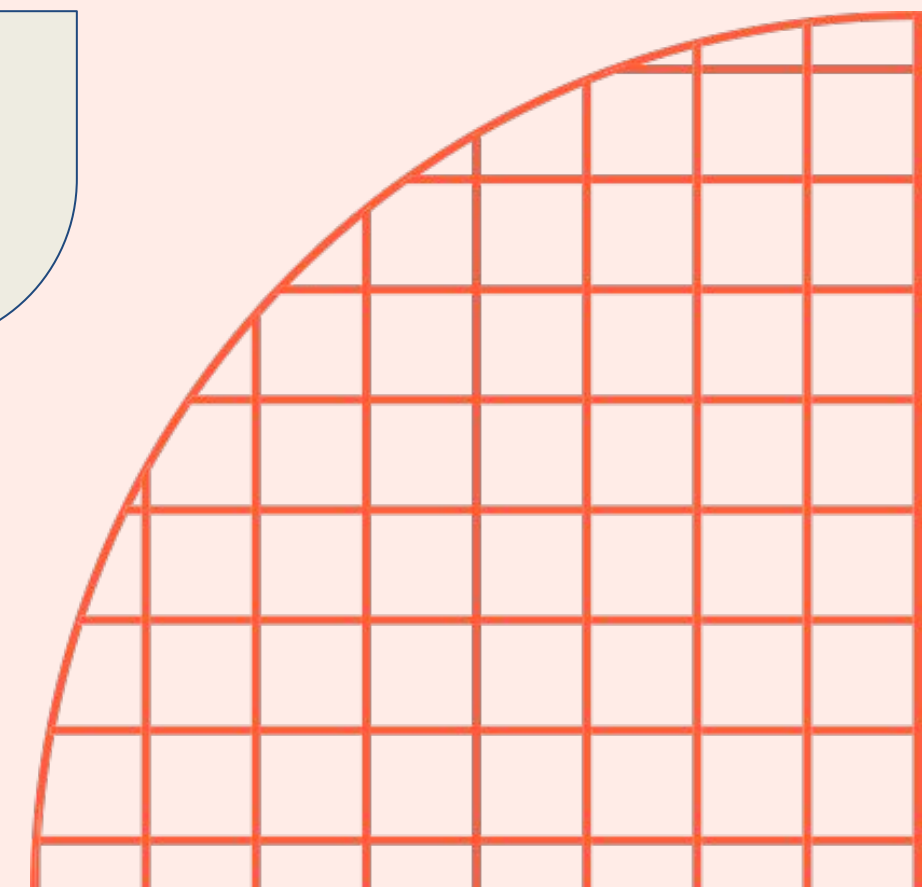
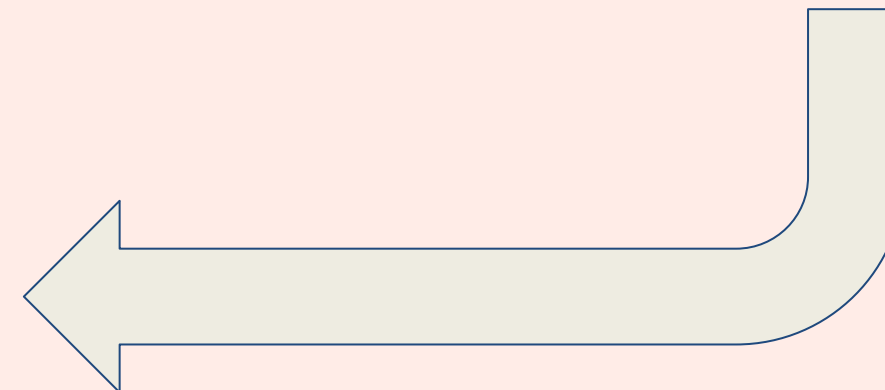


Change Preventer (Divergent Change):

Test logic was mixed with hardcoded locators. If the website UI changed a button ID, we would have to search and change the code in multiple places.

```
private final By cartIcon = By.className("shopping_cart_link");
```

We encapsulated all locators as private final By constants inside their respective Page Objects. Now, a UI change only requires a single update in one file.



Refactorization

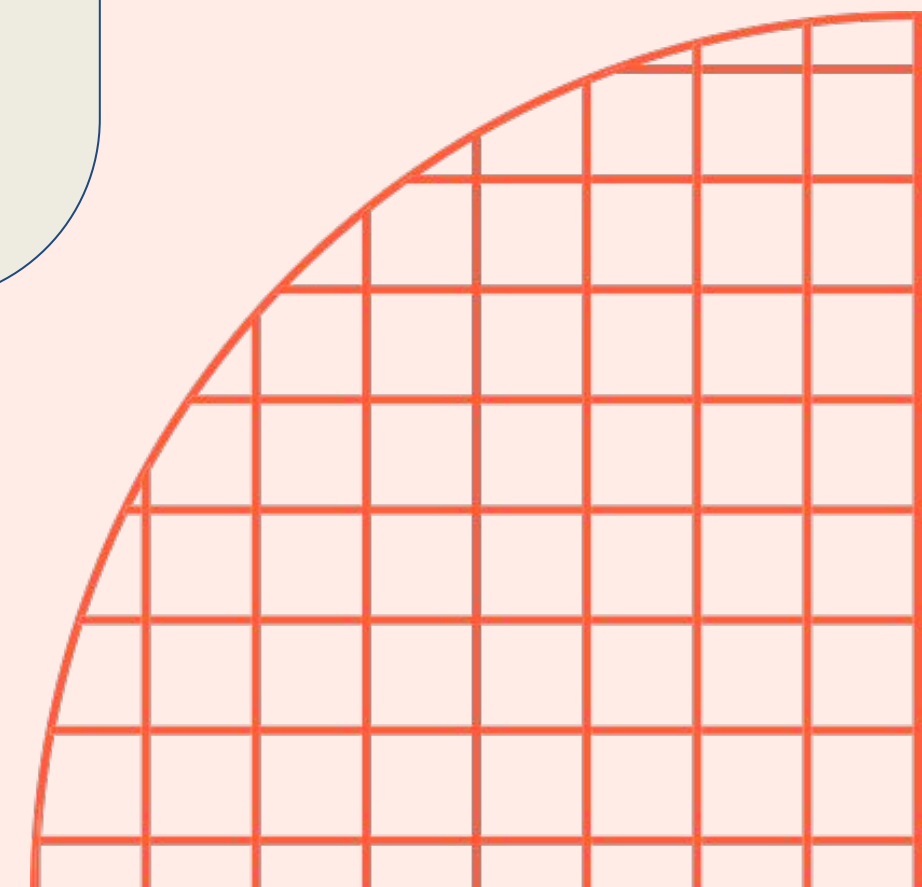
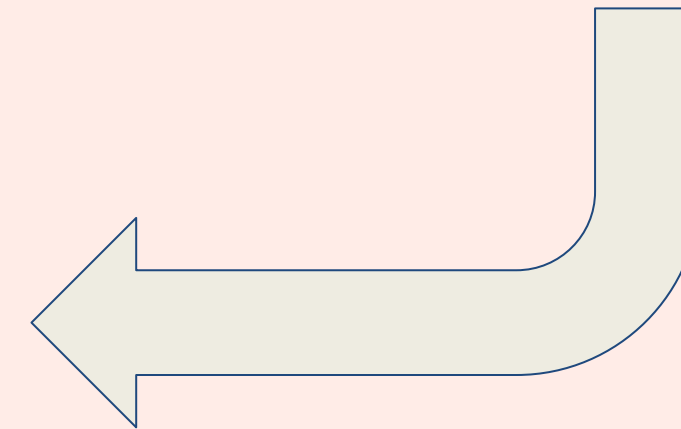
```
// SMELL 8: Duplicate Code (Dispensable) - Teardown should be in @AfterEach  
// Closes the browser session and ends the test  
driver.quit();
```



Dispensable (Duplicate Code):
The driver.quit() command was placed at the end of the test method. If the test failed halfway, the browser would stay open, wasting system resources.

We moved the teardown logic to the BaseTest class using the @AfterEach annotation. This ensures the browser always closes, regardless of whether the test passed or failed.

```
@AfterEach  David Alonso Aguilar  
public void tearDown() {  
    // FIXES SMELL 8: Centralized t  
    // Executed after every test to  
    if (driver != null) {  
        driver.quit();  
    }  
}
```



Refactorization

```
// SMELL 9: Inappropriate Intimacy (Coupler) - Accessing internal logic
// Represents a method that breaks class encapsulation
no usages
public void secretInternalMethod() { System.out.println("Breaking encapsulation"); }
```



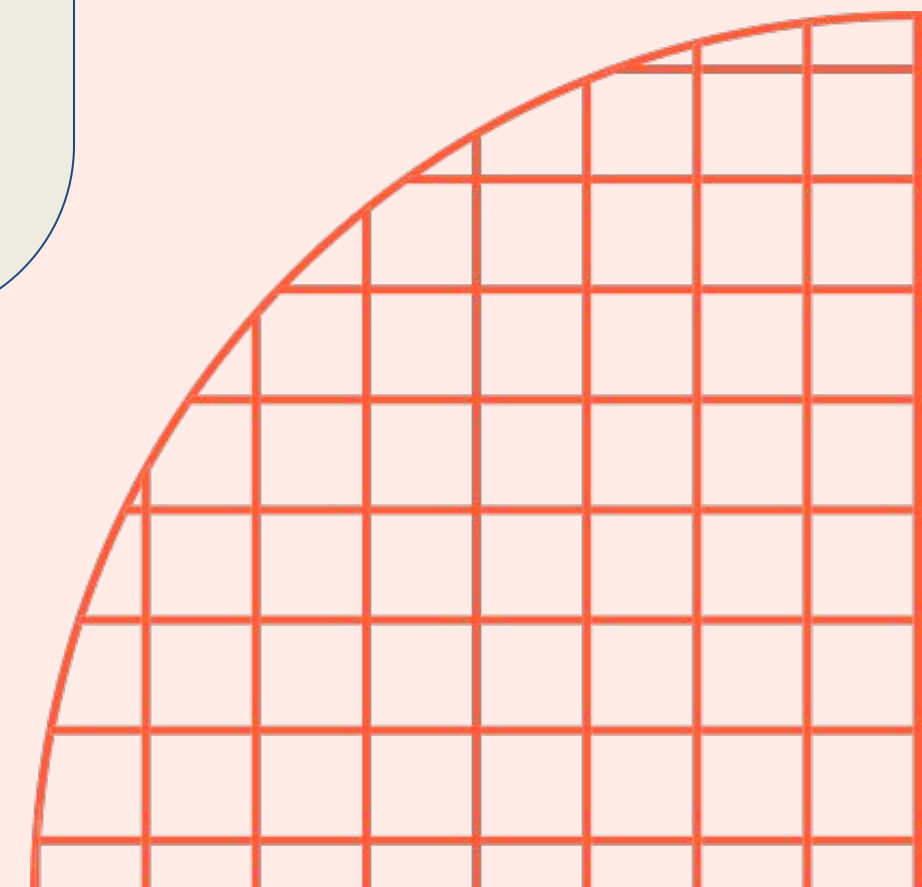
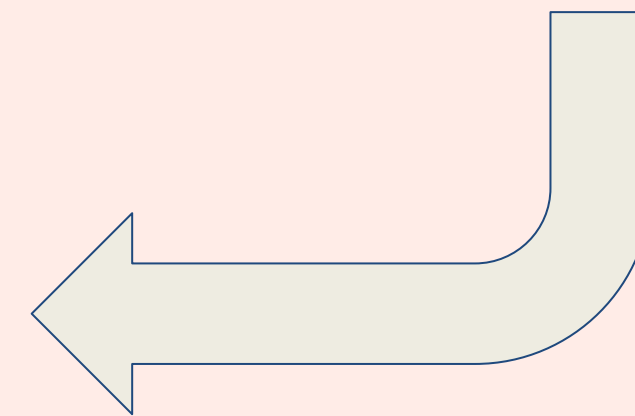
Coupler (Inappropriate Intimacy):

Classes were accessing internal information or methods that should be private, like `secretInternalMethod()`. This breaks the principle of encapsulation.

We applied Access Modifiers (private). We hid the implementation details and only exposed necessary public methods to maintain a clean interface between classes.

```
public class InventoryPage { 3 usages 2
    // Reference to the WebDriver session
    private WebDriver driver; 3 usages

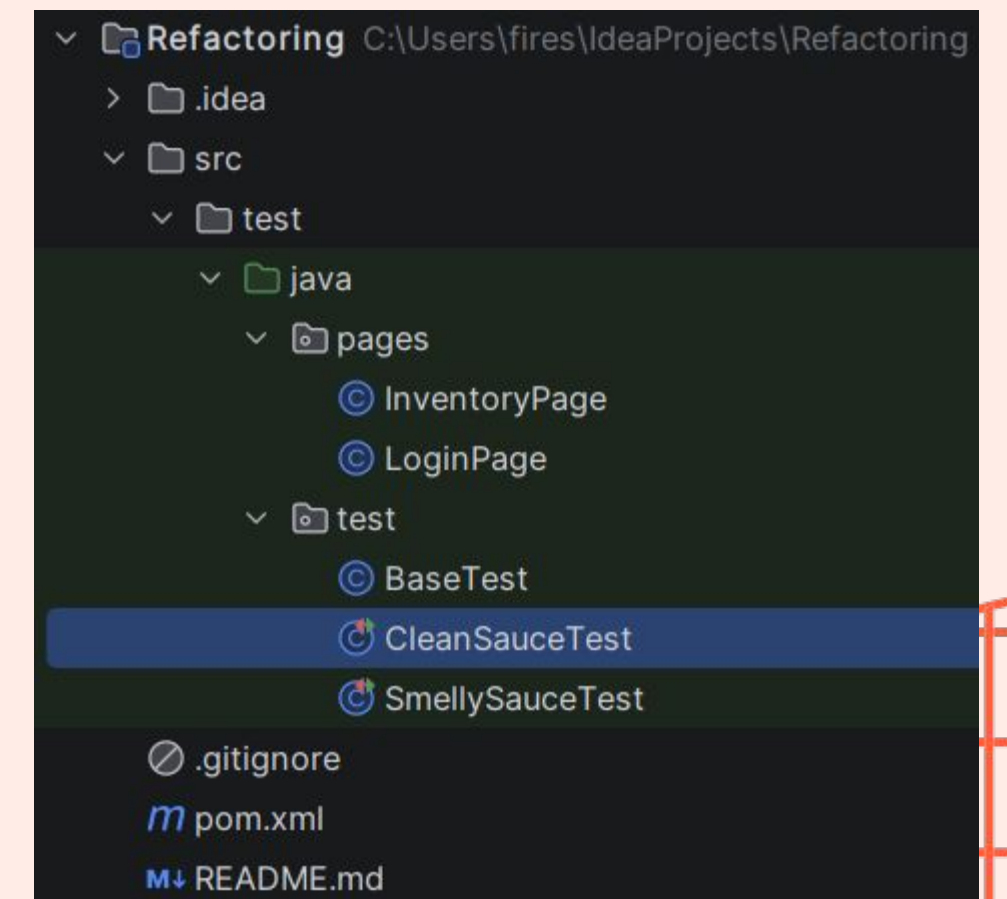
    // FIXES SMELL 7: Centralized location
    // Locates the shopping cart icon to add
    private final By cartIcon = By.className("shopping-cart")
    // Locates the specific button to add item to backpack
    private final By backpackAddButton = By.className("add-to-backpack")
}
```



Refactorization

Bloater (Large Class):
SmellySauceTest was responsible
for the driver, the locators, the
business logic, and the assertions
all at once.

We split the large class into a
hierarchy: BaseTest for
infrastructure, Page Objects for UI,
and CleanSauceTest for the flow.



```
public class SmellySauceTest {  & David Alonso Aguilar +1

    // SMELL 1: Temporary Field (OO Abuser) - Declared at class level but only
    private String tempUser; no usages

    @Test  & David Alonso Aguilar +1
    void completeFlow() throws InterruptedException {
        // SMELL 3: Long Method (Bloater) - This method is too long
        // SMELL 2: Duplicate Code (Dispensable) - Setup should be in @BeforeE
        // Initializes the ChromeDriver and opens the browser window
        WebDriverManager.chromedriver().setup();
        WebDriver driver = new ChromeDriver();
        driver.manage().window().maximize();
        // Navigates the browser to the target web application
        driver.get("https://www.saucedemo.com/");

        // LOGIN
        // Locates input fields by ID and sends credentials
        driver.findElement(By.id("user-name")).sendKeys(...keysToSend: "standard_
        driver.findElement(By.id("password")).sendKeys(...keysToSend: "secret_sau
        driver.findElement(By.id("login-button")).click();
        // SMELL 4: Feature Envy (Coupler) - Test is directly managing the page

        // SMELL 5: Switch Statement (OO Abuser) - Replace later to make the co
        // Selection mechanism to decide which item to add to the cart
        String item = "Backpack";
        switch (item) {
            case "Backpack":
                driver.findElement(By.id("add-to-cart-sauce-labs-backpack")).c
                break;
            case "Bike Light":
                driver.findElement(By.id("add-to-cart-sauce-labs-bike-light"))
                break;
        }

        // SMELL 6: Dead Code (Dispensable) - Unused variable
        String unusedVariable = "I am not used";

        // SMELL 7: Divergent Change (Change Preventer) - Logic is mixed with
        // Clicks the shopping cart icon to view selected items
        driver.findElement(By.className("shopping_cart_link")).click();

        // ASSERTIONS
        // Validates that the URL has changed to the cart page
        assertTrue(driver.getCurrentUrl().contains("cart.html"));
    }
}
```




Conclusion

03

Conclusion

CONCLUSION

This project demonstrates the critical importance of refactoring in automation. We didn't just fix a script, we built a framework.

We successfully eliminated all 10 code smells from the original SmellySauceTest. By implementing the Page Object Model, we separated 'what the test does' from 'how the page works'.

This new architecture significantly reduces maintenance costs. If SauceDemo changes a button ID tomorrow, we only fix it in one place, not in every single test. More importantly, the code is now a professional asset that is easy to read, easy to scale, and ready for a real-world production environment.

